ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA ĐIỆN TỬ VÀ TRÍ TUỆ NHÂN TẠO

BÁO CÁO TỔNG KẾT

CHUYÊN ĐỀ 2

PHÂN TÍCH ĐỘ PHỨC TẠP TÍNH TOÁN VÀ CHI PHÍ PHẦN CỨNG CỦA BỘ LỌC ĐỆ QUY TRONG TINH CHỈNH BẢN ĐỒ TRUYỀN QUA THUỘC BÀI TOÁN KHỬ SƯƠNG ẢNH

Sinh viên thực hiện: Trần Quang Khánh - 106220220
Trương Văn Tú - 106220240
Trần Đình Quang Nhật - 106220265
GVHD: TS. Nguyễn Văn Hiếu
Lớp học phần: 22.44
Lớp sinh hoạt: 22KTMT

TP. Đà Nẵng, 2026

MỤC LỤC

1. Giới thiệu, đặt vấn đề: 2
1.1. Bài toán khử sương ảnh và vai trò của bản đồ truyền qua: 2
1.2. Quy trình khử sương ảnh qua DCP: 2
1.3. Vai trò của khâu tinh chỉnh bản đồ truyền qua và yêu cầu triển khai trên PYNQ-Z2: 3
1.4. Mục tiêu, phạm vi và tiêu chí so sánh FAST2 với GIF65: 3
2. Mô hình bài toán tối ưu và phương pháp tinh chỉnh: 3
2.1. Mô hình tán xạ khí quyển và ước lượng bản đồ truyền qua thô: 3
2.2. Phát biểu bài toán tối ưu tinh chỉnh bản đồ truyền qua qua bài toán phân tích tối ưu lồi: 4
2.3. GIF65 và bài toán hồi quy tuyến tính cục bộ có điều chuẩn: 5
2.4. FAST2 và cơ chế lọc đệ quy giữ biên với nhìn trước một hàng: 6
3. Phân tích độ phức tạp tính toán: 6
3.1. Giả thiết phân tích: 6
3.2. Chi phí tính toán trên mỗi điểm ảnh và trên toàn khung: 7
3.2.1. Chi phí tính toán trên bộ lọc GIF65: 7
3.2.2. Chi phí tính toán trên bộ lọc FAST2: 7
3.2.3. Nhận xét – so sánh: 8
4. Phân tích chi phí phần cứng của FAST2: 8
4.1. Kiến trúc phần cứng theo khối: 8
4.2. Phân tích nhu cầu bộ nhớ và mức sử dụng BRAM: 10
5. Kết quả thực nghiệm và thảo luận: 11
5.1. Tập ảnh, cấu hình thử nghiệm và phương pháp đánh giá: 11
5.2. Chất lượng ảnh sau khử sương theo PSNR, SSIM và so sánh trực quan: 12
5.3. Tài nguyên FPGA của FAST2 sau khi synthesis qua Vitis HLS: 14
5.4. Kiểm chứng đầu ra FAST2 giữa HLS và Python: 14
6. Kết luận và hướng phát triển: 16
Tài liệu tham khảo: 18

1. Giới thiệu, đặt vấn đề:

1.1. Bài toán khử sương ảnh và vai trò của bản đồ truyền qua:

Trong điều kiện có sương mù, sự tán xạ trong khí quyển làm suy giảm ánh sáng từ vật thể đến camera.

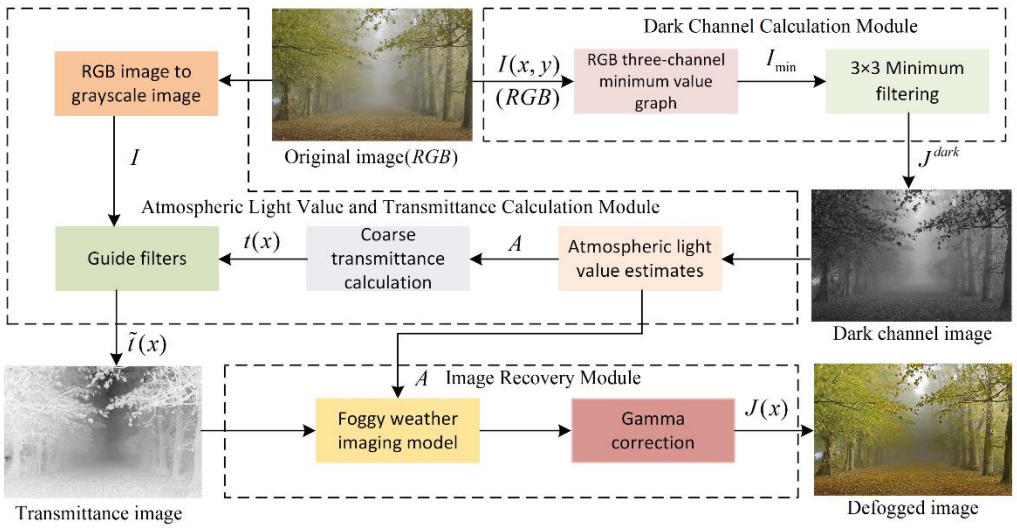
Đồng thời, ánh sáng môi trường bị tán xạ vào đường truyền làm ảnh giảm độ tương phản và khó quan sát chi tiết [1]. Bài toán khử sương ảnh nhằm khôi phục hình ảnh của cảnh trong điều kiện không có sương từ ảnh quan sát được.

Bản đồ truyền qua $t(x)$ biểu diễn tỷ lệ ánh sáng từ vật thể còn truyền được đến camera tại vị trí x [1]. Giá trị $t(x)$ càng nhỏ thì tín hiệu từ vật thể càng bị suy giảm. Do giá trị này được dùng trực tiếp trong bước tái tạo ảnh, đề tài tập trung vào khâu tinh chỉnh bản đồ truyền qua.

1.2. Quy trình khử sương ảnh qua DCP:

Phương pháp Dark Channel Prior (DCP) dựa trên quan sát rằng trong phần lớn các vùng cục bộ của ảnh ngoài trời không có sương, tồn tại điểm ảnh có cường độ rất thấp ở ít nhất một kênh màu RGB [2].

Dựa trên đặc điểm này, phương pháp ước lượng ánh sáng khí quyển và bản đồ truyền qua từ ảnh đầu vào. Như được minh họa ở Hình 1, bản đồ truyền qua thô được tinh chỉnh trước khi dùng để tái tạo ảnh khử sương.



Hình 1: Lưu đồ khử sương dựa trên phương pháp DCP [1]

Theo mô hình tán xạ khí quyển, ảnh quan sát được biểu diễn như sau [2]:

$$I(x) = J(x)t(x) + A(1 - t(x)) \quad (1)$$

Với $I(x)$ là cường độ ảnh có sương, $J(x)$ là cường độ ảnh cần khôi phục, A là ánh sáng khí quyển ở mỗi kênh màu, và $t(x)$ là hệ số truyền qua tại điểm ảnh x.

Hai số hạng trong công thức (1) lần lượt biểu diễn ánh sáng từ vật thể truyền đến camera và ánh sáng môi trường bị tán xạ vào đường truyền.

Sau khi ước lượng A, công thức DCP chuẩn tính bản đồ truyền qua thô bằng phép lấy cực tiểu trên các kênh màu đã chuẩn hóa và trên vùng lân cận $\Omega(x)$ [2] qua công thức (2):

$$\check{t}(x) = 1 - \omega \min_{y \epsilon \Omega(x)} \left(\min_{c} \frac{I^c(y)}{A^c} \right) \quad (2)$$

Hệ số ω ($0 \leq \omega \leq 1$) điều chỉnh mức khử sương. Trong đề tài, nhóm so sánh **GIF65** và **FAST2** tại bước tinh chỉnh $\check{t}(x)$. Sau khi thu được bản đồ truyền qua tinh chỉnh $t(x)$, ảnh đầu ra được tái tạo theo [2] ở công thức (3):

$$J(x) = \frac{I(x) - A}{\max(t(x), t_0)} + A \quad (3)$$

Với ngưỡng ($t_0 > 0$) dùng để giới hạn mẫu số để tránh khuếch đại sai số ở những vùng có hệ số truyền qua quá thấp.

1.3. Vai trò của khâu tinh chỉnh bản đồ truyền qua và yêu cầu triển khai trên PYNQ-Z2:

Khâu tinh chỉnh giúp làm mượt bản đồ truyền qua thô trong từng vùng ảnh và giữ sự thay đổi tại biên vật thể.

Khi thiết kế bộ lọc hướng đến PYNQ-Z2, nhu cầu lưu dữ liệu trung gian ảnh hưởng đến mức sử dụng BRAM. Vì vậy, đề tài khảo sát FAST2 như một phương án giảm nhu cầu bộ đệm so với GIF65.

1.4. Mục tiêu, phạm vi và tiêu chí so sánh FAST2 với GIF65:

Đề tài này chạy hai bộ lọc trên Python với cùng bản đồ truyền qua thô và cùng bước tái tạo ảnh, sau đó so sánh chất lượng ảnh đầu ra bằng PSNR và SSIM.

Phần khảo sát phần cứng dừng ở bước tổng hợp thử bằng Vitis HLS phục vụ cho PYNQ-Z2, với BRAM và LUT là chỉ tiêu tài nguyên chính, vì với bài toán xử lí ảnh truyền thống thì 2 chỉ tiêu tài nguyên này là 2 chỉ tiêu đáng được quan tâm, so với 2 chỉ tiêu còn lại là DSP và FF cho các mô hình Deep Learning hay Machine Learning.

2. Mô hình bài toán tối ưu và phương pháp tinh chỉnh:

2.1. Mô hình tán xạ khí quyển và ước lượng bản đồ truyền qua thô:

Theo mô hình tán xạ khí quyển [2], cường độ ảnh có sương tại điểm ảnh x trên kênh màu c được biểu diễn bởi công thức (4) sau đây:

$$I^c(x) = J^c(x)t(x) + A^c(1 - t(x)) \quad (4)$$

Trong đó, I^c là ảnh có sương, J^c là ảnh cần khôi phục, A^c là ánh sáng khí quyển và $t(x)$ là hệ số truyền qua. Để ước lượng bản đồ truyền qua thô, chương trình trước hết tính kênh tối bằng cách lấy cực tiểu trên ba kênh RGB và trong cửa sổ 15×15 quanh x:

$$D(x) = \min_{y \epsilon \Omega_{15}(x)} \left(\min_{c \in \{R,G,B\}} I^c(y) \right) \tag{5}$$

Sau khi ước lượng $A = (A_R, A_G, A_B)$, chương trình lấy trung bình ba thành phần, làm tròn và chặn dưới tại 1 để thu được giá trị vô hướng A_S. Bản đồ truyền qua thô p được tính theo công thức dưới đây:

$$p(x) = \underset{[0,1]}{\text{clip}} \left(1 - \frac{\omega D(x)}{A_s} \right) \tag{6}$$

Đây là dạng biểu diễn của phép tính khi bỏ qua sai số làm tròn trung gian. Hai bộ lọc GIF65 và FAST2 nhận cùng bản đồ truyền qua thô p. Bước tái tạo ảnh sau lọc sử dụng A^c riêng cho từng kênh màu.

2.2. Phát biểu bài toán tối ưu tinh chỉnh bản đồ truyền qua qua bài toán phân tích tối ưu lồi:

Biến quyết định: Với ảnh có N điểm ảnh, biến cần tìm là một vector có dạng là $t = (t_1, t_2, t_3, \dots, t_N)$. Trong đó:

- t_i $(i = 1, \dots, N)$ là hệ số truyền qua tinh chỉnh tại điểm ảnh i.
- Bản đồ thô p và ảnh dẫn hướng được xem là dữ liệu đã biết.

Các ràng buộc: Hệ số truyền qua biểu diễn tỷ lệ ánh sáng từ vật thể truyền đến camera, nên giá trị tại mỗi điểm ảnh nằm trong khoảng từ 0 đến 1:

$$0 \leq t_i \leq 1 \tag{7}$$

Các trọng số w_{ij} được tính trước từ ảnh dẫn hướng và giữ cố định khi tối ưu. Chúng thỏa $w_{ij} \geq 0$. Tham số làm mượt $\lambda \geq 0$ cũng được chọn trước.

Hàm mục tiêu: Tìm t sao cho tổng sai lệch so với bản đồ thô và chi phí làm mượt giữa các điểm ảnh liền kề là nhỏ nhất:

$$\min_{t \epsilon [0,1]^N} \left[\sum_{i=1}^{N} (t_i - p_i)^2 + \lambda \sum_{\{i,j\} \in \varepsilon} w_{ij} \left(t_i - t_j \right)^2 \right] \tag{8}$$

Ở đây, ε là tập các cặp điểm ảnh liền kề theo chiều ngang hoặc chiều dọc.

Hạng thứ nhất giữ kết quả gần bản đồ thô, hạng thứ hai khuyến khích các điểm ảnh liền kề có hệ số truyền qua gần nhau. Với hai điểm ảnh có màu tương đồng, w_{ij} lớn nên

chênh lệch truyền qua bị phạt mạnh hơn. Qua biên có chênh lệch màu rõ, w_{ij} nhỏ nên mức làm mượt giảm.

Tính lồi:

Hạng bám dữ liệu là hàm bậc hai lồi nghiêm ngặt theo t. Hạng làm mượt là tổng các bình phương với hệ số không âm, nên là hàm lồi. Vì vậy, tổng hai hạng vẫn lồi nghiêm ngặt.

Miền ràng buộc $[0,1]^N$ là tập lồi, đóng và bị chặn, nên bài toán WLS có nghiệm tối ưu duy nhất. Mô hình này dùng để phát biểu mục tiêu tinh chỉnh. GIF65 và FAST2 thực hiện việc tinh chỉnh theo hai cách trình bày như ở phần 2.3 và 2.4 sau đây.

2.3. GIF65 và bài toán hồi quy tuyến tính cục bộ có điều chuẩn:

GIF65 dùng ảnh xám chuẩn hóa g làm ảnh dẫn hướng và bản đồ truyền qua thô p làm dữ liệu cần lọc. Trong cửa sổ Ω_k kích thước 65×65 có tâm tại k, bộ lọc mô hình hóa kết quả cục bộ tại điểm ảnh i theo công thức sau [4]:

$$q_i^{(k)} = a_k g_i + b_k \ \ (i \in \Omega_k) \tag{9}$$

Bước 1: Tính các giá trị trung bình trong cửa sổ. Gọi $n_k$là số mẫu trong Ω_k. Bộ lọc tính trung bình của g, p, tích gp và bình phương g^2:

$$\begin{aligned} \overline{g_k} &= \frac{1}{n_k}\sum_{i\in\Omega_k} g_i\,, \qquad \overline{p_k} = \frac{1}{n_k}\sum_{i\in\Omega_k} p_i, \\ \overline{gp_k} &= \frac{1}{n_k}\sum_{i\in\Omega_k} g_i p_i\,, \qquad \overline{g_k^2} = \frac{1}{n_k}\sum_{i\in\Omega_k} g_i^2 \end{aligned} \tag{10}$$

Bước 2: Tính phương sai và hiệp phương sai. Từ các giá trị trung bình trên, phương sai của ảnh dẫn hướng và hiệp phương sai giữa ảnh dẫn hướng với bản đồ thô được tính như sau:

$$var_k(g) = \overline{g_k^2} - (\overline{g_k})^2, \qquad cov_k(g,p) = \overline{gp_k} - \overline{g_k}.\overline{p_k} \tag{11}$$

Bước 3: Tính hệ số hồi quy cục bộ. GIF65 dùng phương sai và hiệp phương sai để tính hai hệ số:

$$a_k = \frac{cov_k(g,p)}{var_k(g) + \varepsilon}, \qquad b_k = \overline{p_k} - a_k\overline{g_k} \tag{12}$$

Tham số điều chuẩn $\varepsilon = 0.001$ giúp mẫu số luôn dương và hạn chế hệ số a_k tăng quá lớn ở những vùng ảnh ít biến thiên.

Bước 4: Lấy trung bình hệ số và tính đầu ra. Một điểm ảnh có thể thuộc nhiều cửa sổ chồng lấp. Gọi K_i là tập các tâm cửa sổ có chứa điểm ảnh i. Bộ lọc lấy trung bình các hệ số tương ứng:

$$\bar{a_i} = \frac{1}{|K_i|}\sum_{k\in K_i} a_k\,, \qquad \bar{b_i} = \frac{1}{|K_i|}\sum_{k\in K_i} b_i \tag{13}$$

Giá trị truyền qua tinh chỉnh tại điểm ảnh i được tính theo công thức sau:

$$q_i = \bar{a_i} g_i + \bar{b_i} \tag{14}$$

2.4. FAST2 và cơ chế lọc đệ quy giữ biên với nhìn trước một hàng:

FAST2 dùng chênh lệch màu RGB để xác định mức ảnh hưởng giữa hai điểm ảnh liền kề. Phép cập nhật cơ bản trộn giá trị hiện tại u với trạng thái lân cận v theo hệ số α:

$$B_\alpha(u, v) = u + \alpha(v - u) = (1 - \alpha)u + \alpha v \tag{15}$$

Bước 1: Xác định hệ số trộn. Gọi I_i và I_j là hai vector RGB liền kề. Trong mô hình số thực, hệ số trộn được tính theo:

$$d_{ij} = \left\|I_i - I_j\right\|_2, \qquad r_{ij} = \lambda e^{-\frac{d_{ij}}{\sigma}} \tag{16}$$

$$\alpha_{ij} = \frac{2r_{ij}}{1 + 2r_{ij} + \sqrt{1 + 4r_{ij}}} \tag{17}$$

Cấu hình sử dụng $\lambda = 1024$ và $\sigma = 6$. Khi chênh lệch màu lớn, hệ số α_{ij} giảm và trạng thái lân cận ảnh hưởng ít hơn đến kết quả. Trong mô hình số cố định, hệ số trộn được lượng tử hóa và tra từ bảng LUT.

Bước 2: Lọc theo chiều ngang. Áp dụng phép cập nhật B_α như ở công thức (15) theo hai lượt trên mỗi hàng, từ trái sang phải rồi từ phải sang trái. Xét tại một cột cố định, gọi h_y là kết quả lọc ngang tại hàng y.

Bước 3: Lọc theo chiều dọc và hoàn tất hàng trước. Khởi tạo trạng thái hàng đầu $s_0 = h_0$. Khi nhận hàng y, bộ lọc cập nhật trạng thái s_y rồi tính đầu ra q_{y-1} cho hàng trước:

$$s_y = B_{\alpha_y}(h_y, s_{y-1}), \qquad q_{y-1} = B_{\alpha_y}(s_{y-1}, s_y) \tag{18}$$

Ở đây, α_y là hệ số trộn giữa hai điểm ảnh cùng cột thuộc hàng $y - 1$ và y. Với ảnh có H hàng, đầu ra hàng cuối là $q_{H-1} = s_{H-1}$. Kết quả của hàng trước được hoàn tất khi xử lý hàng kế tiếp, nên bước lọc dọc chỉ cần nhìn trước một hàng.

3. Phân tích độ phức tạp tính toán:

3.1. Giả thiết phân tích:

Xét ảnh đầu vào có chiều cao H, chiều rộng W và tổng số điểm ảnh $N = HW$. Phân tích chỉ xét khâu tinh chỉnh bản đồ truyền qua, từ bản đồ thô p đến bản đồ đã lọc q.

Các bước tính kênh tối, ước lượng ánh sáng khí quyển và tái tạo ảnh được dùng chung cho hai phương án nên nằm ngoài phạm vi so sánh này.

Đối với GIF65, ký hiệu $K = 65$ là kích thước cạnh của cửa sổ lọc. Phân tích dựa trên cách tính trung bình cửa sổ bằng ảnh tích phân. Đối với FAST2, phân tích dựa trên hai lượt lọc ngang và bước lọc dọc nhìn trước một hàng. Chi phí tính hệ số trộn được xét riêng với chi phí cập nhật đệ quy.

3.2. Chi phí tính toán trên mỗi điểm ảnh và trên toàn khung:

3.2.1. Chi phí tính toán trên bộ lọc GIF65:

Từ các công thức ở mục 2.3, GIF65 phải thực hiện sáu phép lọc trung bình cửa sổ. Bốn phép đầu tính trung bình của g, p, gp và g^2 để xác định phương sai, hiệp phương sai và hai hệ số a, b. Hai phép còn lại tính trung bình của a, b để tạo đầu ra q.

Nếu cộng trực tiếp mọi mẫu trong cửa sổ, mỗi phép tính trung bình cần xét K^2 mẫu cho một điểm ảnh.

Theo phương pháp, phương trình xây dựng ảnh tích phân rồi lấy tổng cửa sổ từ bốn giá trị ở các góc. Vì vậy, sau khi có ảnh tích phân, chi phí truy vấn một cửa sổ là $O(1)$ trên mỗi điểm ảnh. Các phép tính phương sai, hiệp phương sai, hệ số hồi quy và đầu ra cũng có số phép toán cố định trên mỗi điểm ảnh.

Do ta chọn đệm biên trước khi tạo ảnh tích phân, mỗi lượt xây dựng ảnh tích phân xử lý khoảng $(H + K - 1)(W + K - 1)$ mẫu. Với $K = 65$ cố định, độ phức tạp toàn khung của GIF65 là $O(HW)$. Sáu lượt tính trung bình và các phép toán hồi quy tạo nên phần lớn khối lượng tính toán của bộ lọc.

3.2.2. Chi phí tính toán trên bộ lọc FAST2:

Sau khi có hệ số trộn α theo 2 chiều, mỗi lần cập nhật theo công thức $B_\alpha(u, v) = u + \alpha(v - u) = (1 - \alpha)u + \alpha v$ gồm một phép trừ, một phép nhân và một phép cộng. Trong mô hình số cố định, phép cập nhật còn có thao tác làm tròn và dịch bit.

Ở mỗi lượt lọc ngang, điểm đầu tiên theo hướng quét được dùng để khởi tạo, nên mỗi hàng có $W - 1$ lần cập nhật. Theo chiều dọc, từ hàng thứ hai trở đi, mỗi cột cần hai lần cập nhật: một lần để tính trạng thái của hàng hiện tại và một lần để hoàn tất đầu ra của hàng trước. Số lần cập nhật trên toàn khung được tổng hợp trong Bảng 1.

Bảng 1: Số lần cập nhật đệ quy của bộ lọc FAST2 trên toàn khung

Công đoạn	Số lần cập nhật đệ quy trên toàn khung
Lọc ngang từ trái sang phải	$H(W - 1)$
Lọc ngang từ phải sang trái	$H(W - 1)$
Cập nhật trạng thái dọc và hoàn tất đầu ra của hàng trước	$2W(H - 1)$

Tổng cộng	$4HW - 2H - 2W$

Mỗi lượt ngang có $W - 1$ lần cập nhật trên một hàng vì điểm khởi tạo đã có sẵn. Theo chiều dọc, từ hàng thứ hai trở đi, mỗi cột cần một lần cập nhật trạng thái của hàng hiện tại và một lần hoàn tất đầu ra của hàng trước. Tính trung bình trên toàn khung, số lần cập nhật trên mỗi điểm ảnh là:

$$\frac{4HW - 2H - 2W}{HW} = 4 - \frac{2}{W} - \frac{2}{H} \tag{19}$$

Ngoài các lần cập nhật trên, FAST2 cần tính hệ số trộn cho $H(W - 1)$ cặp điểm ảnh liền kề ngang và $W(H - 1)$ cặp liền kề dọc.

Hệ số của mỗi cặp ngang được dùng cho cả hai lượt lọc ngang. Trong mô hình số cố định, chương trình tính khoảng cách màu RGB rồi dùng khoảng cách đó để tra bảng hệ số.

Tổng số cặp liền kề tỷ lệ với HW, nên khi tính cả bước xác định hệ số trộn, độ phức tạp toàn khung của FAST2 vẫn là $O(HW)$.

3.2.3. Nhận xét – so sánh:

GIF65 và FAST2 đều có độ phức tạp $O(HW)$ theo cách cài đặt qua ipynb (Python notebook). Khác biệt nằm ở các phép toán thực hiện trên từng điểm ảnh. GIF65 xây dựng các ảnh tích phân, tính thống kê cửa sổ và hệ số hồi quy. FAST2 tính hệ số trộn từ chênh lệch màu và thực hiện các lượt cập nhật đệ quy. Big-O cho biết chi phí tăng như thế nào khi kích thước ảnh tăng. Thời gian chạy thực tế trên Python cần được xác định bằng phép đo trên cùng tập ảnh, vì các phép toán mảng của GIF65 và các vòng lặp đệ quy của FAST2 được thực thi theo cách khác nhau.

4. Phân tích chi phí phần cứng của FAST2:

4.1. Kiến trúc phần cứng theo khối:

Theo quy trình, khối FAST2 (theo bộ lọc đệ quy) nhận hai đầu vào gồm bản đồ truyền qua thô p và ảnh RGB dẫn hướng. Bản đồ p đi qua hai lượt lọc ngang, sau đó được lọc dọc với cơ chế nhìn trước một hàng để tạo bản đồ truyền qua tinh chỉnh q. Ảnh RGB được dùng để xác định hệ số trộn cho các cặp điểm ảnh liền kề theo từng chiều.

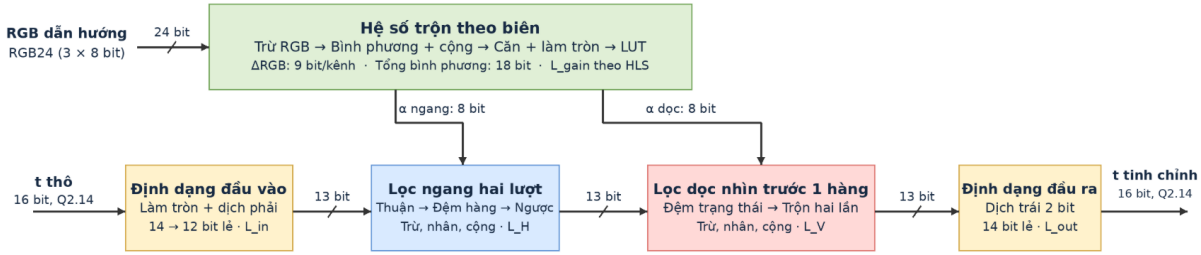
Bảng 2 dưới đây mô tả vai trò của từng khối chức năng:

Bảng 2: Vai trò của các khối chức năng cho FAST2

Khối chức năng	Vai trò
Tính hệ số trộn	Tính độ chênh lệch màu giữa hai điểm ảnh liền kề và tra hệ số từ bảng LUT

Lọc ngang hai lượt	Lọc từng hàng từ trái sang phải, rồi từ phải sang trái. Dữ liệu trung gian của lượt thuận cần được lưu để đọc theo chiều ngược ở lượt sau.
Lọc dọc nhìn trước một hàng	Dùng dữ liệu hàng hiện tại và trạng thái hàng trước để cập nhật trạng thái mới, đồng thời hoàn tất đầu ra cho hàng trước.

Hình 2 dưới đây mô tả kiến trúc phần cứng của bộ lọc FAST2 với hai lượt lọc ngang và lọc dọc nhìn trước một hàng:



Hình 2: Kiến trúc phần cứng của bộ lọc FAST2 với hai lượt lọc ngang và lọc dọc nhìn trước một hàng

Như minh họa trong Hình 2, FAST2 nhận hai luồng dữ liệu từ các khâu trước của hệ thống:

- Luồng thứ nhất là bản đồ truyền qua thô t_{coarse}, có độ rộng 16 bit ở định dạng Q2.14.
- Luồng thứ hai là ảnh RGB dẫn hướng 24 bit, lấy từ ảnh có sương ban đầu. Hai luồng được căn chỉnh theo tọa độ để giá trị truyền qua tại mỗi điểm ảnh được xử lý với hệ số trộn tính từ đúng cặp điểm ảnh RGB lân cận.

Sơ đồ gồm năm khối chức năng. Bốn khối nằm trên đường dữ liệu truyền qua gồm định dạng đầu vào, lọc ngang hai lượt, lọc dọc nhìn trước một hàng và định dạng đầu ra. Khối thứ năm tính hệ số trộn theo biên từ ảnh RGB rồi cấp hệ số α cho cả hai khối lọc.

Đầu ra của FAST2 là bản đồ truyền qua tinh chỉnh t_{coarse}, được đưa về định dạng 16 bit Q2.14 để chuyển sang bước tái tạo ảnh.

Bộ lọc FAST2 được mô tả bằng C++ và tổng hợp bằng Vitis HLS để tạo mã RTL ở dạng Verilog. Trong đó, mã C++ mô tả các phép tính của thuật toán, còn các chỉ thị #pragma HLS hướng dẫn công cụ cách tổ chức phần cứng khi tổng hợp.

Trong thiết kế FAST2, các chỉ thị này có thể được dùng để tạo pipeline cho vòng lặp, cho phép các công đoạn xử lý chồng lấp, cấu hình FIFO và ánh xạ bộ đệm dòng vào BRAM. Bảng 3 dưới đây trình bày cách áp dụng từng chỉ thị.

Bảng 3: Cách áp dụng kĩ thuật pragma trong Vitis HLS

Pragma	Cách áp dụng cho FAST2
PIPELINE II=1	Đặt mục tiêu bắt đầu một vòng lặp xử lý sau mỗi chu kỳ ở những đoạn phù hợp, chẳng hạn vòng lặp tính hệ số trộn. Với vòng lặp đệ quy, trạng thái của lần trước được dùng ở lần sau nên cần kiểm tra II đạt được trong báo cáo tổng hợp.
DATAFLOW và STREAM	Khi tách thiết kế thành các công đoạn, DATAFLOW cho phép các công đoạn xử lý chồng lấp. STREAM cấu hình FIFO cho các kênh truyền tuần tự giữa chúng. Độ sâu FIFO cần đủ để hấp thụ chênh lệch nhịp xử lý giữa các công đoạn.
BIND_STORAGE	Chỉ định kiểu bộ nhớ cho các mảng đệm, chẳng hạn ánh xạ bộ đệm hàng vào BRAM và chọn cấu hình cổng phù hợp với nhu cầu đọc, ghi.

Trong FAST2, FIFO giữa các công đoạn và bộ đệm hàng có hai nhiệm vụ khác nhau. FIFO chuyển mẫu theo thứ tự, còn bộ đệm ngang cho phép đọc lại dữ liệu theo chiều ngược và bộ đệm dọc giữ trạng thái của hàng trước.

Vì vậy, khi phân tích BRAM ở mục 4.2, cần tách dung lượng của các bộ đệm phục vụ thuật toán khỏi dung lượng FIFO dùng để nối các công đoạn.

4.2. Phân tích nhu cầu bộ nhớ và mức sử dụng BRAM:

Như thể hiện trong Hình 2, FAST2 cần lưu dữ liệu trung gian ở hai công đoạn chính là lọc ngang hai lượt và lọc dọc nhìn trước một hàng. Hai công đoạn có cách truy cập bộ nhớ khác nhau, từ đó quyết định dữ liệu cần lưu và cấu hình bộ đệm.

Bộ đệm cho lọc ngang: Trong lượt lọc từ trái sang phải, giá trị đã cập nhật tại mỗi điểm ảnh được lưu theo vị trí cột. Sau khi xử lý đến cuối hàng, lượt lọc từ phải sang trái đọc lại các giá trị theo thứ tự ngược. Do hai lượt duyệt cùng một hàng theo hai hướng, bộ lọc cần vùng nhớ đủ chứa kết quả trung gian của một hàng. Nếu kiến trúc xử lý chồng lấp giữa các hàng, chẳng hạn ghi kết quả lượt thuận của hàng mới trong khi đọc hàng trước cho lượt ngược, có thể cần thêm vùng đệm tùy theo cách tổ chức đọc và ghi.

Bộ đệm cho lọc dọc: Khi xử lý điểm ảnh tại cột x của hàng y, bộ lọc lấy trạng thái tại cùng cột của hàng $y-1$. Từ hai giá trị này, bộ lọc tính trạng thái mới cho hàng y và hoàn tất đầu ra cho hàng $y-1$. Trạng thái mới được giữ lại để xử lý hàng kế tiếp.

Vì vậy, lượng trạng thái cần lưu theo thuật toán là một giá trị cho mỗi cột, tương ứng với W giá trị trên một hàng.

Nhánh tính hệ số trộn theo chiều dọc cũng cần truy cập hai mẫu RGB cùng cột thuộc hai hàng liên tiếp. Nếu thiết kế giữ nguyên một hàng RGB để thực hiện phép tính này,

dung lượng của hàng dẫn hướng được tính riêng. Với bộ đệm chứa W mẫu và mỗi mẫu rộng b bit, dung lượng dữ liệu là:

$$M_{row} = W \times b \ (bit) \tag{20}$$

Trong FAST2, giá trị truyền qua trung gian được biểu diễn với 12 bit phần lẻ. Khi mã hóa giá trị trong đoạn $[0, 1]$, số nguyên lưu trữ có miền từ 0 đến $2^{12} = 4096$.

Vì cần biểu diễn cả giá trị 4096, nên độ rộng tối thiểu là 13 bit. Với ảnh 1920×1080, dung lượng theo từng loại dữ liệu được trình bày trong Bảng 4 như sau:

Bảng 4: Dung lượng dữ liệu của một dòng trong FAST2

Dữ liệu lưu trữ	Số mẫu	Độ rộng mỗi mẫu	Dung lượng
Kết quả trung gian của lượt lọc ngang	1920	13 bit	24960 bit
Trạng thái lọc dọc của hàng trước	1920	13 bit	24960 bit
Ảnh dẫn hướng, nếu lưu nguyên một hàng RGB	1920	24 bit	46080 bit

Nếu mỗi loại trong Bảng 2 được lưu bằng một bộ đệm dòng, tổng dung lượng dữ liệu của ba bộ đệm là 96000 bit, tương đương 12,000 byte. Đây là ước tính theo giả thiết của bảng.

Tuy nhiên, khi đối chiếu với mã HLS, cần thay bằng độ rộng khai báo và số vùng đệm thực tế. Nếu thiết kế có FIFO nối các công đoạn, dung lượng từng FIFO được tính riêng bằng độ sâu FIFO nhân với độ rộng mỗi phần tử.

Mức sử dụng BRAM sau tổng hợp. Bảng 2 cho biết dung lượng dữ liệu, còn số khối BRAM được xác định theo cách Vitis HLS ánh xạ từng mảng. Một bộ đệm có thể cần ghép nhiều khối để đáp ứng đồng thời độ sâu, độ rộng từ nhớ và số cổng truy cập.

Vì vậy, số BRAM ước lượng của FAST2 được lấy từ báo cáo tổng hợp Vitis HLS ở mục 5.3. Khi đối chiếu với GIF65, cần lấy số liệu ở cùng phạm vi tổng hợp, ví dụ so sánh riêng hai khối tinh chỉnh bản đồ truyền qua.

5. Kết quả thực nghiệm và thảo luận:

5.1. Tập ảnh, cấu hình thử nghiệm và phương pháp đánh giá:

Thực nghiệm đánh giá chất lượng ảnh được tiến hành trên hai tập SOTS Indoor và SOTS Outdoor, mỗi tập gồm 500 cặp ảnh có sương và ảnh sạch tham chiếu.

Các ảnh được giữ nguyên kích thước. Trên mỗi ảnh đầu vào, hai phương án sử dụng cùng cách ước lượng ánh sáng khí quyển, cùng bản đồ truyền qua thô và cùng bước tái tạo ảnh. Bộ lọc tinh chỉnh lần lượt là GIF65 và FAST2 nhìn trước một hàng.

Cấu hình dùng chung gồm DCP với cửa sổ 15×15, hệ số $\omega = 0.82$, mức bù $+492$ theo biểu diễn Q14 sau lọc và ngưỡng truyền qua $t_0 = 0.10$ khi tái tạo ảnh. GIF65 sử

dụng cửa sổ 65×65 với $\varepsilon = 0.001$. FAST2 sử dụng $\lambda = 1024$, $\sigma = 6$ và trạng thái có 12 bit phần lẻ.

PSNR và SSIM được tính giữa ảnh sau khử sương và ảnh sạch tham chiếu, sau đó lấy trung bình theo từng tập. Thời gian thực thi được đo riêng cho khâu tinh chỉnh bản đồ truyền qua trên Python. Với mỗi ảnh, chương trình chạy làm nóng một lượt, đo ba lượt và lấy trung vị. Phép đo này loại trừ thời gian đọc ảnh, tính DCP, ước lượng ánh sáng khí quyển, tái tạo ảnh và tính các chỉ số đánh giá.

5.2. Chất lượng ảnh sau khử sương theo PSNR, SSIM và so sánh trực quan:

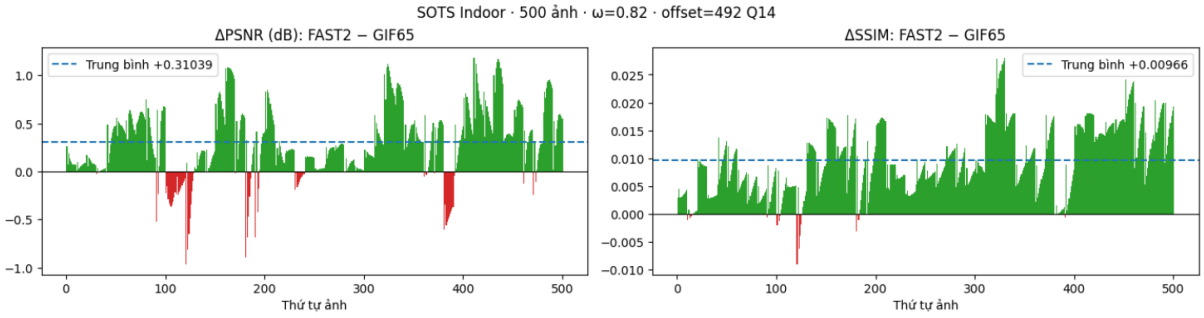
Bảng 5 mô tả kết quả so sánh GIF65 và FAST2 trên SOTS:

Bảng 5: Kết quả so sánh GIF65 và FAST2 trên SOTS

Tập thử	Bộ lọc	Số lượng ảnh	PSNR trung bình (dB)	SSIM trung bình	Thời gian lọc trung vị (ms/ảnh)
SOTS Indoor	GIF65	500	19,77689	0,83128	94,79
SOTS Indoor	FAST2	500	20,08728	0,84094	62,26
SOTS Outdoor	GIF65	500	21,29070	0,91479	81,79
SOTS Outdoor	FAST2	500	20,98415	0,91338	55,41

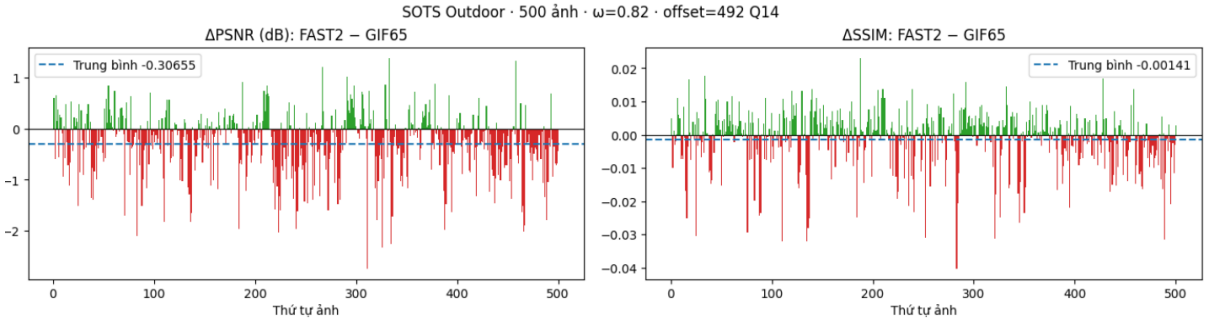
Trên SOTS Indoor, FAST2 tăng PSNR trung bình 0,31039 dB và tăng SSIM trung bình 0,00966 so với GIF65. Hình 3 cho thấy 421/500 ảnh cải thiện đồng thời cả hai chỉ số. Trên SOTS Outdoor, FAST2 giảm PSNR trung bình 0,30655 dB và giảm SSIM trung bình 0,00141. Mặc dù chênh lệch trung bình nhỏ, có 69/500 ảnh Outdoor giảm hơn 1 dB, cho thấy mức chênh lệch phụ thuộc vào từng cảnh.

Hình 3 và Hình 4 biểu diễn chênh lệch PSNR và SSIM giữa FAST2 và GIF65 trên từng ảnh thuộc hai tập SOTS Indoor và SOTS Outdoor. Với mỗi ảnh, chênh lệch được tính bằng giá trị của FAST2 trừ giá trị của GIF65. Cột màu xanh biểu thị FAST2 đạt chỉ số cao hơn, cột màu đỏ biểu thị FAST2 đạt chỉ số thấp hơn. Đường nét đứt biểu thị chênh lệch trung bình trên 500 ảnh của mỗi tập.



Hình 3: Chênh lệch PSNR, SSIM trên Python giữa FAST2 và GIF65 trên SOTS Indoor

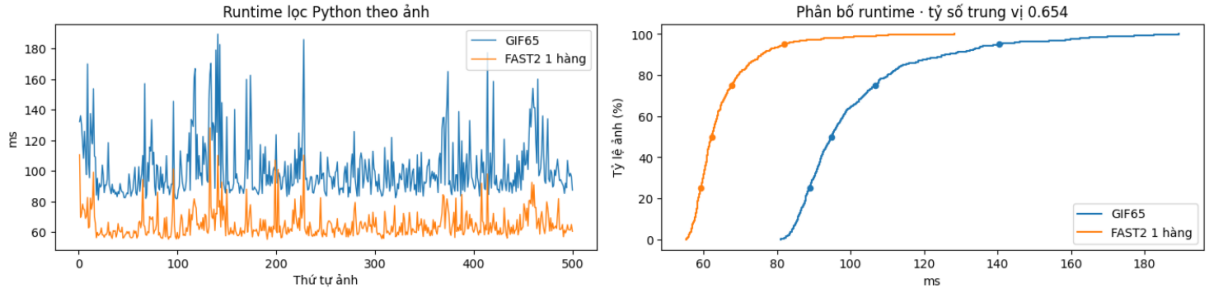
Trên tập SOTS Indoor, FAST2 đạt kết quả trung bình cao hơn GIF65 ở cả hai chỉ số, với PSNR tăng 0,31039 dB và SSIM tăng 0,00966. Có 421 trên 500 ảnh cải thiện đồng thời PSNR và SSIM. Một số ảnh vẫn có chênh lệch âm, nhưng mức giảm PSNR lớn nhất là khoảng 0,97 dB.



Hình 4: Chênh lệch PSNR, SSIM trên Python giữa FAST2 và GIF65 trên SOTS Outdoor

Trên tập SOTS Outdoor, PSNR và SSIM trung bình của FAST2 thấp hơn GIF65 lần lượt 0,30655 dB và 0,00141. Mặc dù chênh lệch trung bình nhỏ, có 69 trên 500 ảnh giảm PSNR hơn 1 dB, trong đó mức giảm lớn nhất khoảng 2,75 dB. Kết quả cho thấy mức chênh lệch chất lượng giữa hai bộ lọc thay đổi theo từng cảnh ngoài trời.

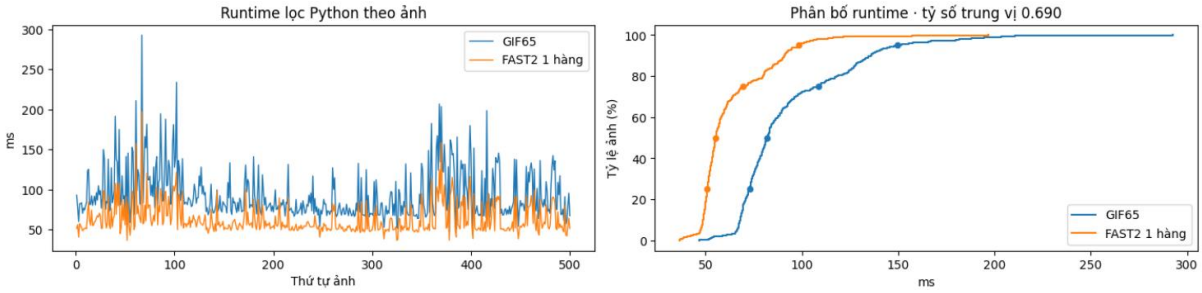
Bên cạnh chất lượng ảnh sau khử sương, thực nghiệm còn đo thời gian thực thi của riêng khâu tinh chỉnh bản đồ truyền qua trên Python. Hình 5 và Hình 6 trình bày kết quả trên hai tập SOTS Indoor và SOTS Outdoor. Trong mỗi hình, đồ thị bên trái biểu diễn thời gian lọc theo từng ảnh, còn đồ thị bên phải cho biết tỷ lệ ảnh được xử lý trong một ngưỡng thời gian nhất định. Đường phân bố nằm càng về bên trái thì bộ lọc càng cần ít thời gian để xử lý cùng một tỷ lệ ảnh.



Hình 5: Thời gian lọc của GIF65 và FAST2 trên tập SOTS Indoor

Trên tập SOTS Indoor, thời gian lọc trung vị của FAST2 là 62,26 ms/ảnh, so với 94,79 ms/ảnh của GIF65. Trung vị của tỷ số thời gian FAST2/GIF65 tính theo từng ảnh là 0,654, tương ứng với mức giảm khoảng 34,6%. Đường phân bố của FAST2 nằm về

bên trái đường của GIF65, cho thấy lợi thế về thời gian xuất hiện trên nhiều ảnh trong tập thử.



Hình 6: Thời gian lọc của GIF65 và FAST2 trên tập SOTS Outdoor

Trên tập SOTS Outdoor, thời gian lọc trung vị của FAST2 là 55,41 ms/ảnh, so với 81,79 ms/ảnh của GIF65. Trung vị của tỷ số thời gian theo từng ảnh là 0,690, tương ứng với mức giảm khoảng 31,0%. Thời gian lọc biến động giữa các ảnh, nhưng đường phân bố của FAST2 vẫn dịch về phía thời gian thấp hơn.

Cả hai bộ lọc đều có độ phức tạp $O(HW)$ theo phân tích ở phần 3. Trong cấu hình Python đã thử nghiệm, FAST2 có thời gian thực thi khâu lọc thấp hơn trên cả hai tập ảnh. Các số đo này dùng để so sánh hai cách cài đặt trên Python. Tuy nhiên, độ trễ và thông lượng FPGA cần được đánh giá riêng từ kết quả HLS.

5.3. Tài nguyên FPGA của FAST2 sau khi synthesis qua Vitis HLS:

Thiết kế FAST2 được tổng hợp bằng Vitis HLS cho thiết bị đích của PYNQ-Z2. Bảng 6 trình bày tài nguyên ước lượng sau tổng hợp.

Trong đó, BRAM là chỉ tiêu được quan tâm chính vì kiến trúc FAST2 cần lưu dữ liệu trung gian để thực hiện lượt lọc ngang ngược và giữ trạng thái của hàng trước khi lọc dọc, như đã phân tích ở mục 4.2.

Bảng 6: Tài nguyên ước lượng của khối FAST2 và toàn bộ hệ thống khử sương

Tài nguyên	Riêng khối FAST2	Toàn bộ hệ thống
BRAM18K	21 (7,50%)	71 (25,36%)
DSP	12 (5,45%)	20 (9,09%)
FF	2.594 (2,44%)	8.920 (8,38%)
LUT	3.610 (6,79%)	14.478 (27,21%)

Theo Bảng 4, khối FAST2 sử dụng 21 BRAM_18K, chiếm 7,50% tài nguyên BRAM của thiết bị đích và chiếm 29,58% trong tổng số BRAM mà hệ thống khử sương sử dụng 21/71. Toàn bộ hệ thống sử dụng 71 BRAM_18K, tương ứng 25,36% tài nguyên BRAM của thiết bị. Số liệu của khối FAST2 đã nằm trong số liệu toàn hệ thống.

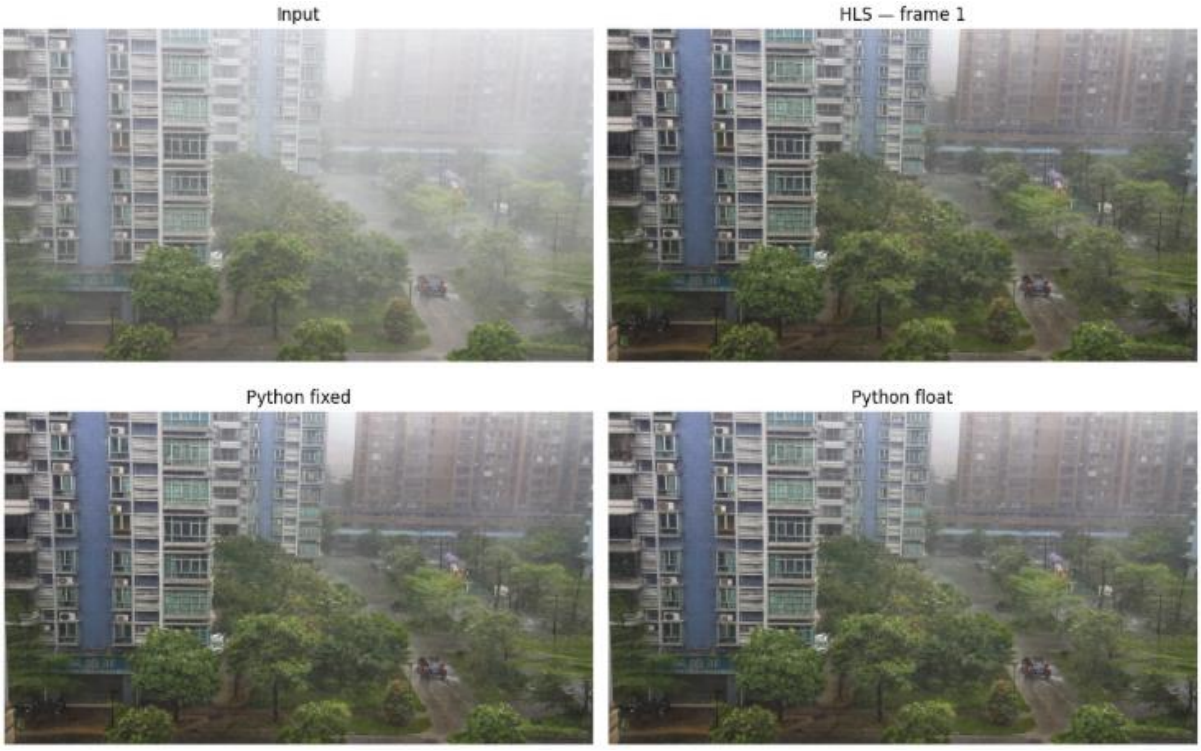
5.4. Kiểm chứng đầu ra FAST2 giữa HLS và Python:

Đầu ra RGB của FAST2 từ HLS được đối chiếu với mô hình Python trên một ảnh kích thước 1920×1080. Phép thử sử dụng khung thứ hai khi đưa cùng một ảnh vào hai lần, với giá trị ánh sáng khí quyển $A = (245, 246, 250)$.

Bảng 7: Sai khác đầu ra RGB giữa HLS và các mô hình Python

Cặp đối chiếu	MAE (mức cường độ)	Sai khác tuyệt đối lớn nhất	Tỷ lệ điểm ảnh trùng khớp cả ba kênh
HLS và Python số cố định	0	0	100%
HLS và Python số thực	0,0660	5	83,52%

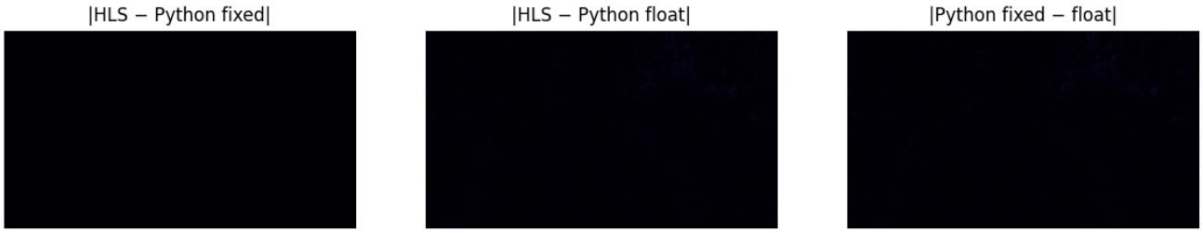
Kết quả cho thấy đầu ra HLS khớp với mô hình Python số cố định tại mọi điểm ảnh trên khung đã kiểm tra. Khi đối chiếu với mô hình Python số thực, sai khác trung bình là 0,0660 mức cường độ. Đây là phép kiểm chứng độ khớp đầu ra giữa các cách triển khai.



Hình 7: So sánh ảnh đầu vào và đầu ra khử sương của FAST2 giữa HLS và Python

Hình 7 trình bày ảnh có sương đầu vào và ba kết quả xử lý tương ứng từ HLS, Python số cố định và Python số thực. Sau khi khử sương, các chi tiết ở mặt tiền tòa nhà và vùng cây xanh rõ hơn so với ảnh đầu vào. Ba ảnh đầu ra có màu sắc và độ tương phản gần

nhau khi quan sát trực tiếp. Để kiểm tra mức độ trùng khớp ở từng điểm ảnh, sai khác giữa các đầu ra được trình bày ở Hình 8.



Hình 8: Bản đồ sai khác tuyệt đối RGB giữa các đầu ra FAST2

Tại mỗi điểm ảnh, bản đồ biểu diễn trung bình sai khác tuyệt đối trên ba kênh RGB. Ở cặp HLS và Python số cố định, bản đồ có màu đen đồng nhất, phù hợp với kết quả MAE bằng 0 và 100% điểm ảnh trùng khớp cả ba kênh trên khung Full HD đã thử. Khi so sánh HLS với Python số thực, sai khác xuất hiện rải rác, với MAE khoảng 0,066 và sai khác tuyệt đối lớn nhất là 5 mức cường độ trên một kênh. Hai bản đồ đối chiếu với Python số thực giống nhau vì đầu ra HLS và Python số cố định trùng khớp trên khung này.



Hình 9: Bản đồ truyền qua thô và sau tinh chỉnh FAST2 trên cùng vùng ảnh

Hình 9 so sánh bản đồ truyền qua thô với hai bản đồ sau tinh chỉnh bằng FAST2 trong Python số cố định và số thực. Sau lọc, các biến thiên nhỏ trong vùng tán cây giảm đi, trong khi ranh giới chính giữa tán cây và hậu cảnh vẫn quan sát được.

Hai bản đồ sau lọc có hình dạng gần nhau. Trên toàn khung, giá trị truyền qua trung bình của mô hình số cố định thay đổi từ 0,58287 trước lọc xuống 0,58215 sau lọc. Hình 9 minh họa tác động của FAST2 lên dữ liệu trung gian trong Python, còn Hình 8 kiểm chứng độ khớp của ảnh RGB đầu ra giữa HLS và Python.

6. Kết luận và hướng phát triển:

Đề tài đã phân tích bài toán tinh chỉnh bản đồ truyền qua trong quy trình khử sương DCP và khảo sát hai bộ lọc GIF65 và FAST2. Mô hình bình phương tối thiểu có trọng số được dùng để mô tả yêu cầu bám theo bản đồ truyền qua thô và làm mượt giữ biên. Với cách cài đặt trong notebook, cả hai bộ lọc đều có độ phức tạp $O(HW)$ đối với ảnh kích thước $H \times W$.

Kết quả trên 1.000 ảnh SOTS cho thấy FAST2 cải thiện PSNR trung bình 0,310 dB và SSIM trung bình 0,00966 so với GIF65 trên tập Indoor. Trên tập Outdoor, hai chỉ số

giảm lần lượt 0,307 dB và 0,00141. Trong cấu hình Python đã thử nghiệm, mức giảm thời gian lọc trung vị theo từng ảnh là 34,6% trên Indoor và 31,0% trên Outdoor.

Kết quả tổng hợp bằng Vitis HLS ước lượng khối FAST2 sử dụng 21 BRAM18K, tương ứng 7,50% tài nguyên BRAM của thiết bị đích trên PYNQ-Z2. Toàn bộ hệ thống khử sương sử dụng 71 BRAM_18K. Khi đối chiếu trên một khung ảnh Full HD, đầu ra HLS và mô hình Python số cố định trùng khớp ở cả ba kênh RGB tại mọi điểm ảnh.

Kết quả đạt được:

- Mô hình hóa bài toán tinh chỉnh bản đồ truyền qua và phân tích độ phức tạp tính toán của GIF65 và FAST2.
- So sánh chất lượng ảnh và thời gian lọc trên Python với 1.000 ảnh SOTS. FAST2 có thời gian lọc trung vị thấp hơn GIF65 trên cả hai tập Indoor và Outdoor.
- Xây dựng kiến trúc FAST2, phân tích nhu cầu bộ đệm và tổng hợp bằng Vitis HLS. Riêng khối FAST2 sử dụng 21 BRAM_18K, tương ứng 7,50% tài nguyên BRAM của thiết bị đích.
- Kiểm chứng đầu ra HLS trùng khớp với mô hình Python số cố định ở cả ba kênh RGB trên khung Full HD đã thử.

Hạn chế:

- Chất lượng ảnh của FAST2 thay đổi theo từng cảnh. Trên SOTS Outdoor, 69 trong 500 ảnh có PSNR thấp hơn GIF65 trên 1 dB.
- Phép đối chiếu đầu ra HLS với Python số cố định mới được thực hiện trên một khung ảnh Full HD.
- Kết quả tài nguyên hiện có chỉ dành cho thiết kế sử dụng FAST2, nên mức chênh lệch BRAM so với GIF65 chưa được định lượng.

Hướng phát triển:

- Phân tích các cảnh ngoài trời có mức giảm PSNR lớn và khảo sát lại tham số FAST2 để cải thiện chất lượng ảnh.
- Mở rộng kiểm chứng đầu ra trên nhiều ảnh và chuỗi khung liên tiếp, sau đó đo độ trễ và thông lượng khi triển khai trên PYNQ-Z2.
- Tổng hợp GIF65 và FAST2 trên cùng thiết bị đích, cùng ràng buộc xung nhịp và cùng phạm vi khối lọc để định lượng mức chênh lệch BRAM.

Tài liệu tham khảo:

[1] L. Wang, Z. Luo, and L. Gao, “An Improved Dark Channel Prior Method for Video Defogging and Its FPGA Implementation,” Symmetry, vol. 17, no. 6, art. 839, 2025.

[2] K. He, J. Sun, and X. Tang, “Single Image Haze Removal Using Dark Channel Prior,” IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. 33, no. 12, pp. 2341–2353, 2011.

[3] Z. Farbman, R. Fattal, D. Lischinski và R. Szeliski, “Edge-Preserving Decompositions for Multi-Scale Tone and Detail Manipulation,” 2008.

[4] K. He, J. Sun và X. Tang, “Guided Image Filtering,” 2013.

Link github báo cáo của nhóm:

https://github.com/sharksvn2804/-HK1-2026-2027-CD2-Final-Report---KTMT